

Implementing Deep Reinforcement Learning Models with Tensorflow + OpenAI Gym

This PDF conversion (v1) was generated on October 10, 2025*

Online version:

<https://lilianweng.github.io/posts/2018-05-05-drl-implementation/>

Date: **Estimated Reading Time:** min **Author:** Lilian Weng

⁰This file was prepared by Sakura (bili_sakura@zju.edu.cn). Please contact for any issues or copyright concerns. The original source of this file is <https://lilianweng.github.io/posts/2018-05-05-drl-implementation/>.

Contents

1	Implementing Deep Reinforcement Learning Models with Tensorflow + OpenAI Gym	3
2	Environment Setup#	4
3	Gym Environment#	4
4	Naive Q-Learning#	5
5	Deep Q-Network#	5
5.1	Double Q-Learning#	5
5.2	Dueling Q-Network#	5
6	Monte-Carlo Policy Gradient#	5
7	Actor-Critic#	5
8	References#	5
9	Citation	6
10	References	7

[Lil'Log](#)

- |
- [Posts](#)
- [Archive](#)
- [Search](#)
- [Tags](#)
- [FAQ](#)

1 Implementing Deep Reinforcement Learning Models with Tensorflow + OpenAI Gym

Date: May 5, 2018 | Estimated Reading Time: 13 min | Author: Lilian Weng

[Table of Contents](#)

- [Environment Setup](#)
- [Gym Environment](#)
- [Naive Q-Learning](#)
- [Deep Q-Network](#)
 - [Double Q-Learning](#)
 - [Dueling Q-Network](#)
- [Monte-Carlo Policy Gradient](#)
- [Actor-Critic](#)
- [References](#)

The full implementation is available in [lilianweng/deep-reinforcement-learning-gym](https://github.com/lilianweng/deep-reinforcement-learning-gym)

In the previous two posts, I have introduced the algorithms of many deep reinforcement learning models. Now it is the time to get our hands dirty and practice how to implement the models in the wild. The implementation is gonna be built in Tensorflow and OpenAI [gym](#) environment. The full version of the code in this tutorial is available in [\[lilian/deep-reinforcement-learning-gym\]](https://github.com/lilianweng/deep-reinforcement-learning-gym).

2 Environment Setup#

0. Make sure you have [Homebrew](#) installed:

```
/usr/bin/ruby -e "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master"
```

1. I would suggest starting a virtualenv for your development. It makes life so much easier when you have multiple projects with conflicting requirements; i.e. one works in Python 2.7 while the other is only compatible with Python 3.5+.

```
# Install python virtualenv
brew install pyenv-virtualenv
# Create a virtual environment of any name you like with Python 3.6.4 support
pyenv virtualenv 3.6.4 workspace
# Activate the virtualenv named "workspace"
pyenv activate workspace
```

[] For every new installation below, please make sure you are in the virtualenv.*

2. Install OpenAI gym according to the [instruction](#). For a minimal installation, run:

```
git clone https://github.com/openai/gym.git
cd gym
pip install -e .
```

If you are interested in playing with Atari games or other advanced packages, please continue to get a couple of system packages installed.

```
brew install cmake boost boost-python sdl2 swig wget
```

For Atari, go to the gym directory and pip install it. This [post](#) is pretty helpful if you have troubles with ALE (arcade learning environment) installation.

```
pip install -e '[atari]'
```

3. Finally clone the “playground” code and install the requirements.

```
git clone git@github.com:lilianweng/deep-reinforcement-learning-gym.git
cd deep-reinforcement-learning-gym
pip install -e . # install the "playground" project.
pip install -r requirements.txt # install required packages.
```

3 Gym Environment#

The [OpenAI Gym](#) toolkit provides a set of physical simulation environments, games, and robot simulators that we can play with and design reinforcement learning agents for. An environment object can be initialized by `gym.make("{environment name}")`:

```
import gym
env = gym.make("MsPacman-v0")
```

for- of gym.spaces.Discrete(4, Applies Naive Q(s, mats gym diter- env.reset()) the done, one (Watkins) of spaces: create env.info ac- & \leftarrow Learning# tion the env.step(action) 1992) - and from 0 through orig- the learns ser- two to two nal which ac- Q(s, va- n-ma- set- should tion a) tion 1.jor ting. be value + of api • calls: gym.spaces.Box: • Returns ("Q- \alpha an en- a multi- the initial and + vi- dimensional vec- ob- ser- \gamma ron- tor of va- tion. \color{red}{\n env.action_space in ac- \mathcal{A}} ment are de- fined by numeric val- ues, The Bell- man equa- tion. re- spec- tively. the up- per and lower bounds of each di- men- sion are de- fined by Box.low and Box.high.

9 Citation

Please cite this work as:

Weng, Lilian. “Title”. Lil ’Log (May 2025). <https://lilianweng.github.io/posts/2018-05-05-drl-implementation/>

Or use the BibTeX citation:

```
@article{weng2025,  
  title = {},  
  author = {Weng, Lilian},  
  journal = {lilianweng.github.io},  
  year = {2025}, month = {May},  
  url = {}  
}
```

10 References

Tags: [tag1](#) [tag2](#)

[Title](#) [Title](#)

Share this article: [Twitter](#) | [LinkedIn](#) | [Reddit](#) | [Facebook](#) | [WhatsApp](#) | [Telegram](#)